

My AI: Self-Hosted Local Workspace

Technical Documentation & Architecture Guide

Table of Contents

- . [1. System Overview](#)
 - . [1.1 Architectural Responsibilities](#)
- . [2. Key Features](#)
- . [3. LM Studio Server & Hardware Setup](#)
 - . [3.1 Server Configuration in LM Studio](#)
 - . [3.2 Verify Server Connectivity](#)
- . [4. Installation and Environment Setup](#)
 - . [4.1 Prerequisites](#)
 - . [4.2 Clone & Set Up Virtual Environment](#)
 - . [4.3 Install Dependencies](#)
 - . [4.4 Initialize the Application](#)
- . [5. UI Walkthrough](#)
 - . [5.1 Authentication \(/login & /register\)](#)
 - . [5.2 Chat Workspace \(/\)](#)
- . [6. Document and Vision Data Flow](#)
 - . [6.1 Multi-Format Document Processing](#)
 - . [6.2 Vision Processing Flow](#)
- . [7. Database Architecture](#)
- . [8. API Reference](#)
- . [9. Running and Deployment](#)
 - . [9.1 Development Mode](#)
 - . [9.2 Production Mode \(Gunicorn + Nginx\)](#)
- . [10. Troubleshooting](#)

A privacy-focused, multi-user web chat workspace integrating local Large Language Models (LLMs) and Vision-Language Models (VLMs) served via LM Studio's OpenAI-compatible inference server.

This platform uses a Flask backend alongside an embedded SQLite database to provide session authentication, chat persistence, multi-modal vision analysis, and contextual document extraction across PDF, DOCX, CSV, JSON, TXT, and Markdown files.

1. System Overview

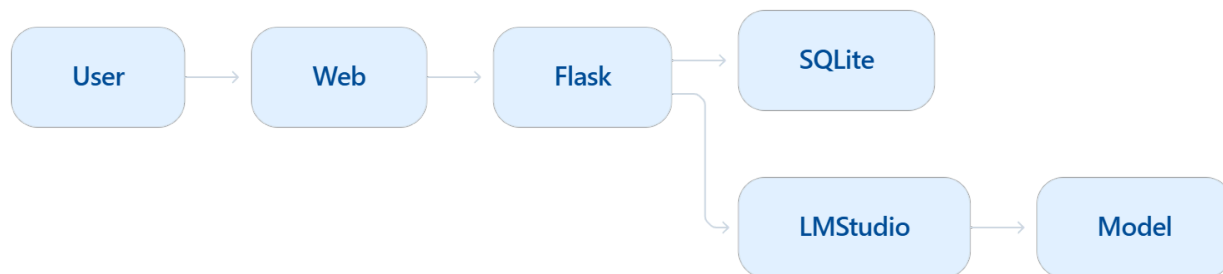


Figure 1: High-level System Architecture Diagram

Architectural Responsibilities

- **Presentation Layer:** A dark-themed web UI ("My AI") providing prompt suggestion chips, conversation tracking, and attachment controls for images and documents.
- **Application Controller (app.py):** Manages session auth, extracts document strings, structures conversation histories, and redirects model queries.
- **Storage Layer (chat.db):** Tracks users, conversations, and chronological message exchanges with cascading foreign keys.
- **Inference Host (LM Studio):** Serves local model weights on http://127.0.0.1:1234/v1 without external telemetry.

2. Key Features

- **Local Compute:** Prompts, parsed documents, and generated completions stay entirely on local storage and memory.
- **Dual Model Pipeline:**
 - **Text Ingestion & Generation:** Powered by qwen/qwen3-coder-30b for logic, summarization, and coding tasks.
 - **Visual Scene Understanding:** Routed to gemma-4-12b via Base64 image payloads.
- **Document Extraction Engine:**
 - **PDF Documents:** Page-by-page text extraction via pypdf.
 - **Word Files (.docx):** Paragraph and table grid parsing with cell delimiters (|) via python-docx.
 - **Structured Data:** JSON formatting and CSV row serialization.
 - **Code & Notes:** Direct extraction of .txt and .md files.
- **Operational Guards:**
 - Max 20 MB upload threshold.
 - Truncates extracted documents to 100,000 characters to prevent model context window overflow.
- **Thread Auto-Titling:** Automatically names new conversations from the first 45 characters of the initial prompt or document name.

3. LM Studio Server & Hardware Setup

The backend communicates with LM Studio via its developer local inference server.

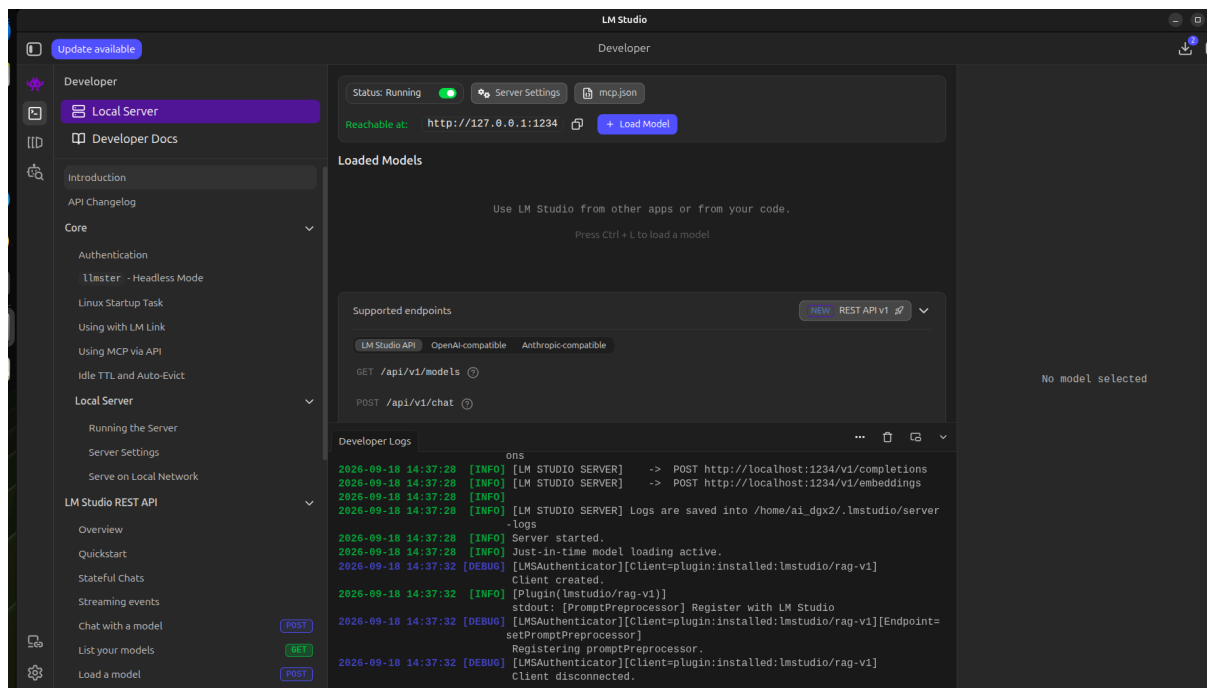


Figure 2: LM Studio Developer / Local Server Interface

3.1 Server Configuration in LM Studio

1. Open LM Studio and navigate to the **Developer / Local Server** tab (icon <->).
2. Click **Start Server** and verify it binds to 127.0.0.1 on port 1234.
3. Check the developer logs for the following entries:
 - o [INFO] Server started.
 - o -> POST http://localhost:1234/v1/chat/completions
4. Open the top model selection dropdown and load qwen/qwen3-coder-30b (or your chosen model).
5. Set the model context length according to your hardware capacity (8,192 to 32,768 tokens recommended).

3.2 Verify Server Connectivity

```
curl http://127.0.0.1:1234/v1/models
```

4. Installation and Environment Setup

4.1 Prerequisites

- Python 3.10, 3.11, or 3.12
- LM Studio (running locally on port 1234)
- Linux, macOS, or Windows (WSL2 recommended for Windows)

4.2 Clone & Set Up Virtual Environment

Clone the project repository and navigate into the project directory:

```
git clone https://github.com/Alexjmathew7/CHAT_APPLICATION
cd CHAT_APPLICATION
```

Create and activate a virtual environment:

- **Linux / Ubuntu:**
python3 -m venv venv
source venv/bin/activate
- **Windows:**
python -m venv venv
venv\Scripts\activate

4.3 Install Dependencies

```
pip install flask pypdf python-docx openai werkzeug
# Or using requirements.txt if available:
pip install -r requirements.txt
```

4.4 Initialize the Application

Install and open LM Studio, load text and vision models, and start the local server at <http://127.0.0.1:1234/v1>. Ensure model IDs configured in app.py match those in LM Studio.

```
python app.py
```

5. UI Walkthrough

5.1 Authentication (/login & /register)

A centered modal interface handles user sessions:

- Salted password verification via `check_password_hash` prevents plaintext credential leaks.
- Form validation checks minimum length (username ≥ 3 chars, password ≥ 6 chars).

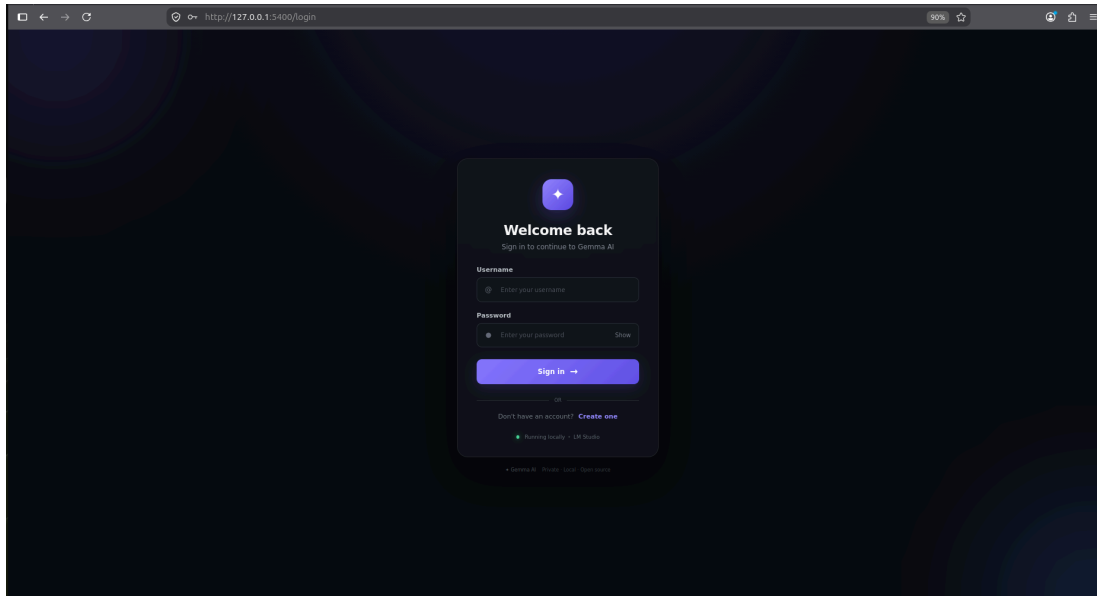


Figure 3: Authentication / Login Interface

5.2 Chat Workspace (/)

The primary chat interface provides access to conversations, prompt shortcuts, and model attachments:

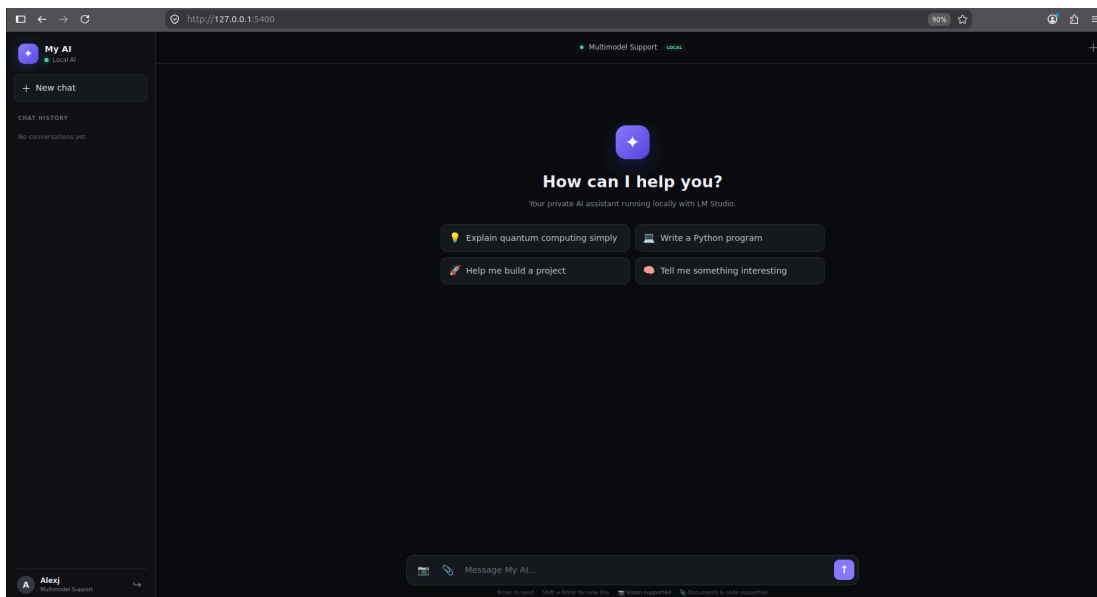


Figure 4: Main Chat Workspace Interface

- **Conversation Sidebar:** Lists active chat threads ordered by updated_at.
- **Prompt Shortcuts:** Starter cards populate the message buffer with standard requests.
- **Input Actions:**
 -  **Camera Icon:** Triggers the system file dialog for image upload, encoded as Base64.
 -  **Paperclip Icon:** Uploads files to /api/upload (PDF, DOCX, CSV, JSON, TXT, MD).

- ↑ **Send Button**: Dispatches the context payload to /chat.

6. Document and Vision Data Flow

6.1 Multi-Format Document Processing

When a document is uploaded, text is extracted, sanitized, and injected directly into the LLM system prompt context:

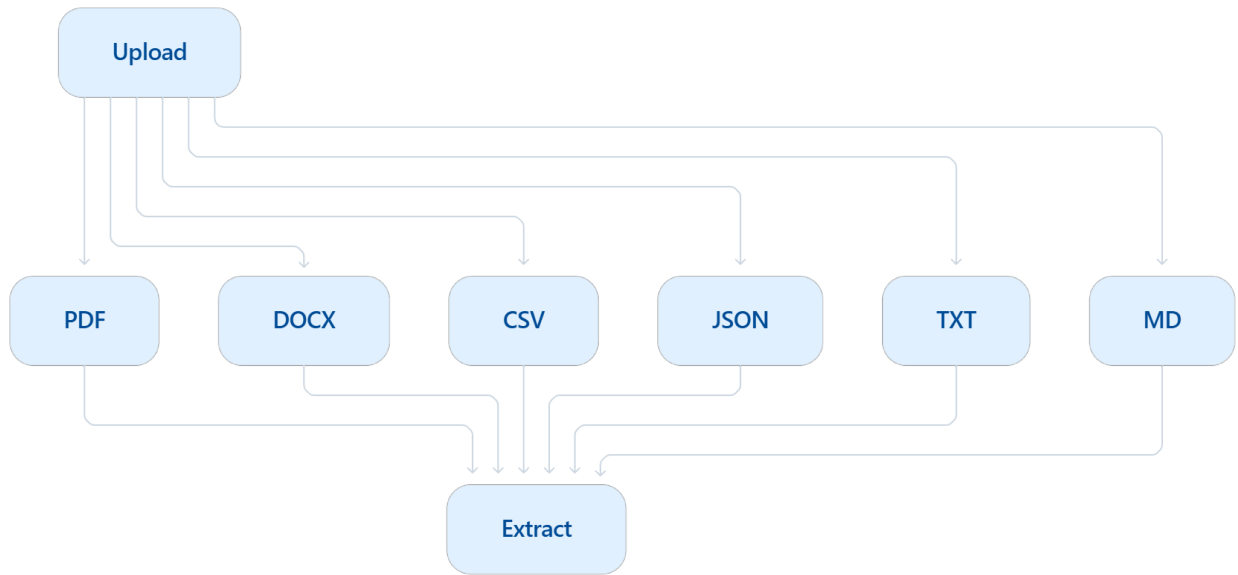


Figure 5: Document Upload & Processing Diagram

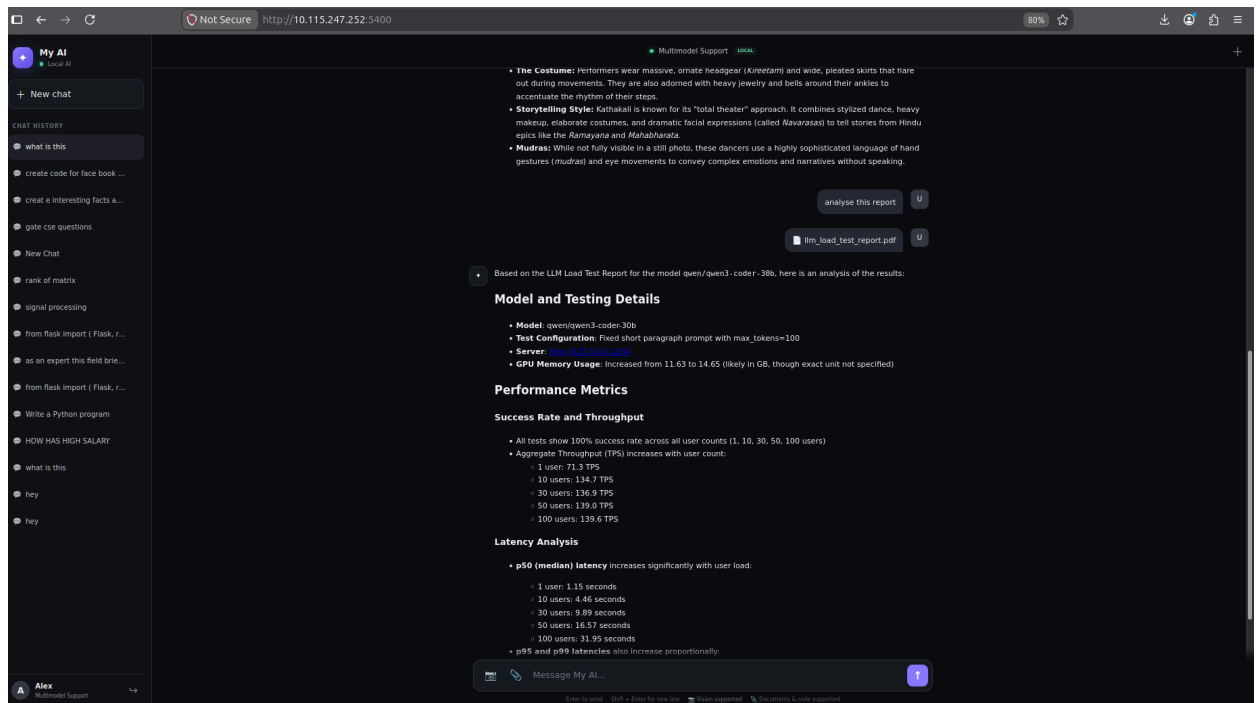


Figure 6: Document Analysis Chat Output

6.2 Vision Processing Flow

Images are sent to /chat as base64-encoded Data URLs, switching model routing to the configured vision model:

```
vision_content = [  
    {"type": "text", "text": user_message or "Describe and analyze this image."},  
    {"type": "image_url", "image_url": {"url": "data:image/jpeg;base64,..."}}  
]
```

```
api_messages.append({"role": "user", "content": vision_content})  
# Dispatched to VISION_MODEL (e.g., gemma-4-12b)
```

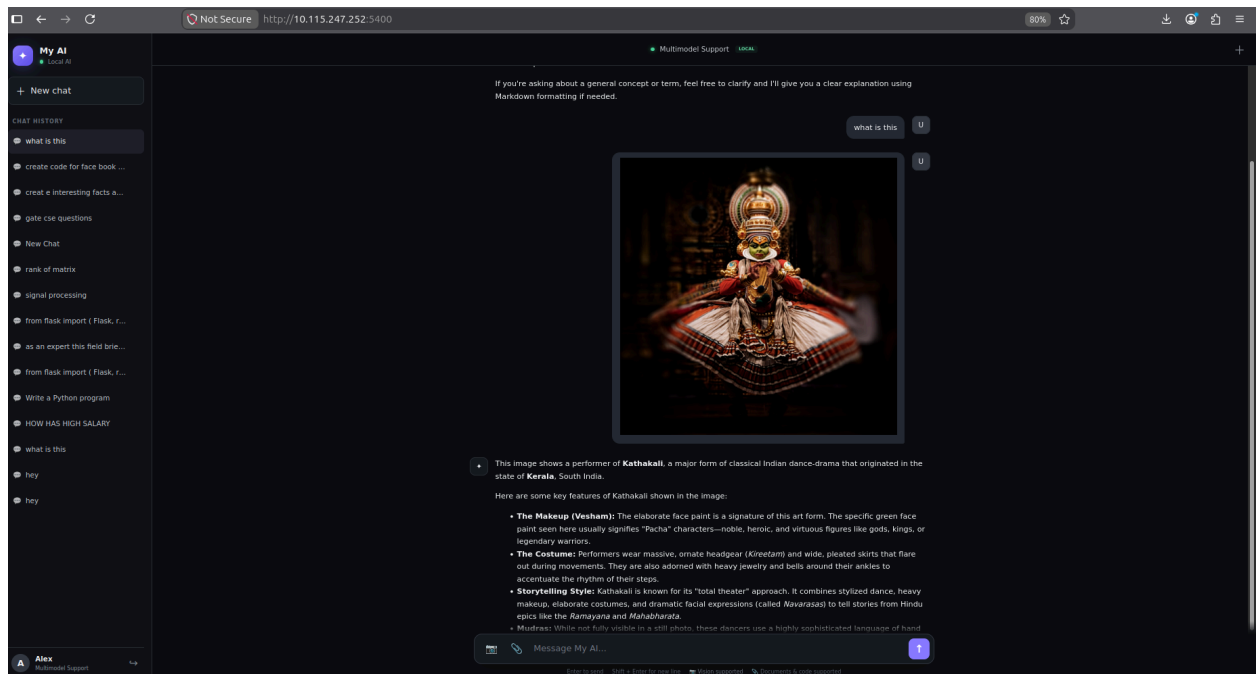


Figure 7: Multimodal Vision Processing Workspace Interface

7. Database Architecture

The SQLite schema (chat.db) uses cascading foreign keys:

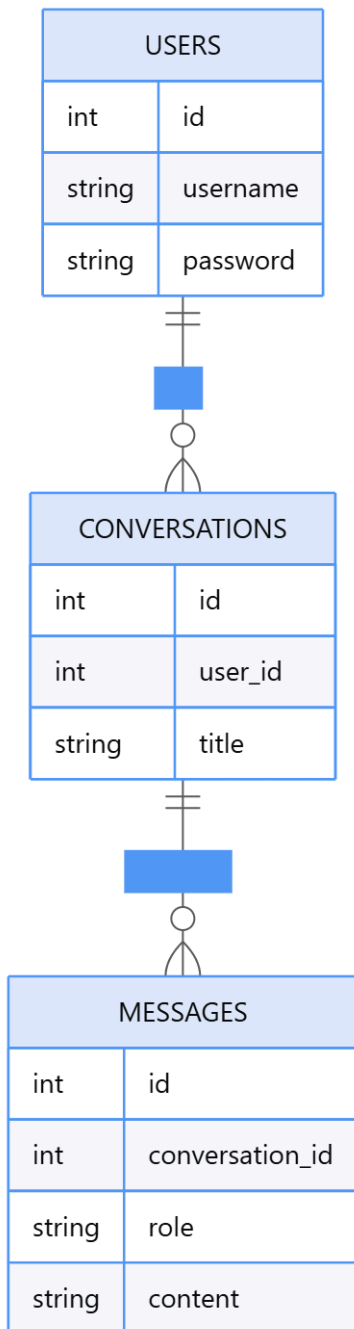


Figure 8: Database ER Schema Diagram

PRAGMA foreign_keys ON;

```

CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY
  AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT
  CURRENT_TIMESTAMP
);
  
```

```

CREATE TABLE IF NOT EXISTS
conversations (
  id INTEGER PRIMARY KEY
  AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  title TEXT NOT NULL DEFAULT 'New
  Chat',
  created_at TIMESTAMP DEFAULT
  CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT
  CURRENT_TIMESTAMP,
  FOREIGN KEY(user_id) REFERENCES
  users(id) ON DELETE CASCADE
);
  
```

```

CREATE TABLE IF NOT EXISTS messages (
  id INTEGER PRIMARY KEY
  AUTOINCREMENT,
  conversation_id INTEGER NOT NULL,
  role TEXT NOT NULL,
  content TEXT NOT NULL,
  image_path TEXT,
  created_at TIMESTAMP DEFAULT
  CURRENT_TIMESTAMP,
  FOREIGN KEY(conversation_id)
  REFERENCES conversations(id) ON
  DELETE CASCADE
);
  
```

8. API Reference

8.1 Core API Endpoints

Method	Endpoint	Description	Auth Required
POST	/register	Create account with form data (username, password)	No
POST	/login	Authenticate user and assign session cookie	No
GET	/logout	Terminate session and clear cookies	Yes
GET	/api/conversations	Retrieve all conversations for the authenticated user	Yes
POST	/api/conversations	Create a new conversation thread	Yes
GET	/api/conversations/<id>	Fetch all messages within a specific conversation	Yes
DELETE	/api/conversations/<id>	Delete a conversation and cascade-delete its messages	Yes
POST	/api/upload	Parse and extract text from an uploaded document	Yes
POST	/chat	Submit messages, images, or document context	Yes

		to the model	
--	--	--------------	--

9. Running and Deployment

9.1 Development Mode

```
python3 app.py
```

The server listens on 0.0.0.0:5000 (Local access at <http://127.0.0.1:5000> and LAN access at http://SERVER_IP:5000).

9.2 Production Mode (Gunicorn + Nginx)

For multi-user or production setups, run Flask behind a production WSGI server:

10. Troubleshooting

- **LM Studio Connection Refused (127.0.0.1:1234):** Check the **Developer** tab in LM Studio to confirm the server status is **Running** on port 1234.
- **Vision Model Returns Error:** Ensure VISION_MODEL in app.py matches the exact model ID displayed in LM Studio.
- **Empty Assistant Generation:** Occurs when context length limits are reached or during GPU out-of-memory errors. Check LM Studio's developer log console for raw engine warnings.
- **Document Extraction Returns Empty:** Scanned or rasterized image PDFs without embedded OCR text layers will return empty strings; run them through an OCR pipeline before uploading.